IDEAS

> **Potential future ideas, not definitive plans.** Nothing in this file is committed, scheduled or agreed. The current design and milestone state are in `DESIGN.md`; this file only preserves directions worth exploring later.

Separate interactive layout engine

LilyPond is the right final engraver but the wrong latency budget for note entry: a page render can take tens of seconds, while composing needs keystroke-rate feedback. The goal is an interactive notation surface that feels immediate, while LilyPond remains the authoritative renderer and the `.ly` file remains the only score file.

Both flavors below are Swift-only. No JavaScript, no VexFlow, no second score document. Glyphs can come from a LilyPond-generated atlas or an SMuFL/Emmentaler font; geometry and layout stay portable, with AppKit/Core Graphics/Core Text confined to `PondsideUI`.

Shared constraints:

- **The `.ly` file is the source of truth.** The interactive engine is a view and an editor over the CST, never a sidecar score format.
- **LilyPond remains the final engraver.** The actual LilyPond render is the canonical page; the engine only shortens the feedback loop.
- **Notation-first, not a literal piano roll.** Time-on-x/pitch-on-y direct manipulation is useful, but a DAW-style piano roll cannot represent beams, slurs, cues, `\lexicon` entries or graphic notation. Use the interaction model, not the visual language.
- **Line breaks must be shared, not guessed.** The engine and LilyPond need a common layout map for system/page breaks, staff positions and horizontal anchors. If they disagree, the preview is lying.
- **Purely graphic notation is opaque.** Arbitrary LilyPond/Scheme stencils (for example the electronics staff in `Tests/Scores/large-graphic/electronics.ly` and `graphics.ily`) are not reimplemented in Swift. Their rendered images are composited as cached layers, keyed by source hash and LilyPond version, and invalidated when the source changes.

Flavor 1 — isolated notation lens

The page stays the primary surface. A temporary notation lens appears for the selected event or system, and Escape dismisses it.

- The lens shows one system with no page breaks: a single continuous strip or loupe, using standard glyphs, staff size and typography.
- It opens on edit, not every selection: selection highlights the system, the first note/duration edit opens the lens, or an explicit key does.
- Editing only happens by keyboard. Mouse selects where in the score to start editing; but click to add note functionality is removed, to avoid confusion. There is only one way to edit notes: through the

pop-up editor lens (which opens automatically on an edit key-stroke) via the keyboard / menu items.
- Experiment with look and feel. Let's try a modal overlay look to start. Blur the main score (which still renders in the background, with instantaneous glyph action untouched) and place lens over it. Smoothly scroll the lens to always center on the cursor. no mouse action effects the lens.
- Edits still write minimal `TextEdit`s into the `.ly`; Escape only closes the lens. A background render then updates the page and cross-fades. If the render before the escape key is hit, the lens closes automatically.
- Because the lens is temporary, it does not need to solve global line/page breaking. That is its main advantage.
- Limits: it only helps while open, can feel modal, and still needs a semantic event selection plus a fallback (placeholder?) for arbitrary graphic elements.

Flavor 2 — interactive engine as the main surface

Most composing happens directly in a fast Swift notation engine. LilyPond renders the final page in the background. The final page can still be viewed in the app, but it is read-only; editing happens in the engine or, for advanced cases, in the source.

- The engine handles ordinary notation directly: note entry, pitch, duration, rests, simple beams and slurs. It must use the same glyph shapes and staff metrics as LilyPond so the composing surface still looks like Pondside, even if local spacing is approximate.
- Purely graphic elements are overlaid as images. The engine does not try to reproduce the Scheme drawings in `graphics.ily`; it composites their cached LilyPond-rendered images at the positions given by the layout map. A whole-staff overlay is acceptable where the graphic layer dominates.
- Line breaks are synced through a shared layout map:
  - Explicit `\break` and `\pageBreak` commands in the source are hard constraints. The large-graphic fixture already does this in `Tests/Scores/large-graphic/score.ly`.
  - Automatic breaks come from the last LilyPond render. The engine lays out against that map until an edit invalidates it.
  - When an edit would change the map, the engine shows a provisional reflow and asks LilyPond for the new layout; the final page and the map update together. The engine must never silently diverge from the render.
  - If the engine's own breaks are meant to be final, it can emit explicit `\break`/`\pageBreak`s into the `.ly`; otherwise LilyPond remains the authority and the engine follows the confirmed map.
- The final artifacts remain the `.ly` file and its actual LilyPond render. The engine writes minimal edits to the source; it does not create a second score file or a second source of truth.
- Limits: this is a real notation layout engine, not a small feature. It needs enough spacing, beaming, accidental, slur and context support to be credible; graphic overlays go stale until LilyPond re-renders them; and line-break authority is the central design problem. True <100 ms arbitrary editing is the payoff; a full LilyPond-quality engine is the cost.

Open questions

- Which notation subset must the engine support before it is useful, and what falls back to source editing or the full render?

- Is the engine or LilyPond the authority for automatic line/page breaks? Explicit breaks are easy; automatic breaks need a confirmation loop.
- How much WYSIWYG fidelity is required inside the engine versus only in the final render?
- How should graphic overlays be invalidated and re-rendered without a visible stale frame?
- Can the glyph atlas be generated from the installed LilyPond version at runtime, so 2.24 and 2.26 output stay consistent without bundling fonts?